

Tổng Hợp Kiến Thức

Server Components (SC)

Định nghĩa

- Các React component được **render hoàn toàn trên server**.
- Mặc định trong App Router (thư mục `app`).
- Kết quả: HTML hoặc RSC Payload được gửi xuống client.

Đặc điểm

- **Chạy trên server**: Không gửi JavaScript xuống client (trừ khi render Client Components con).
- **Không có state hoặc lifecycle**: Chỉ render một lần cho mỗi request.
- **Hỗ trợ `async/await`**: Fetch dữ liệu trực tiếp (ví dụ: truy vấn database).
- **An toàn**: Thích hợp cho logic backend nhạy cảm (không lộ ra frontend).
- **Hạn chế**:
 - Không dùng được React hooks (`useState`, `useEffect`, ...).
 - Không dùng được browser APIs (`window`, `localStorage`).
 - Không dùng được event handlers (`onClick`, `onChange`).

Khi nào sử dụng

- Render phía server để tối ưu SEO.
- Truy cập trực tiếp tài nguyên backend (database, file system).
- Nội dung tĩnh hoặc ít thay đổi theo tương tác người dùng.
- Tối ưu bundle size bằng cách giữ dependencies lớn trên server.
- Cải thiện hiệu suất (giảm JavaScript gửi xuống client).

Lấy dữ liệu (Data Fetching)

- Sử dụng `async/await` trực tiếp trong component.
- Ví dụ:

```
async function fetchPosts() {
  const res = await fetch('<https://jsonplaceholder.typicode.com/posts>');
  if (!res.ok) throw new Error('Failed to fetch posts');
  return res.json();
}

export default async function BlogPage() {
  const posts = await fetchPosts();
  return (/* render posts */);
}
```

- **Tối ưu**: Dùng `Promise.all` để fetch song song:

```
const [posts, users] = await Promise.all([fetchPosts(), fetchUsers()]);
```

Xử lý URL

- **Dynamic Routes**: Truy cập qua prop `params`.

```
export default async function BlogDetailPage({ params }) {
  const { blogId } = await params;
```

```
// Fetch dữ liệu bằng blogId
}
```

- **Search Params:** Truy cập qua prop `searchParams` .

```
export default async function Home({ searchParams }) {
  const { keyword = 'không có từ khóa' } = await searchParams;
  return <p>Từ khóa: {keyword}</p>;
}
```

Client Components (CC)

Định nghĩa

- Các React component được render trên trình duyệt sau **hydration**.
- **Hydration:** Quá trình biến HTML tĩnh từ server thành tương tác bằng cách gắn event listeners và state.

Đặc điểm

- **Đánh dấu:** `"use client";` ở đầu file.
- **Pre-render trên server**, sau đó hydrate trên client.
- **JavaScript được gửi xuống client** để hỗ trợ tương tác.
- **Hỗ trợ:**
 - State (`useState`), effects (`useEffect`), và React hooks.
 - Browser APIs (`document` , `localStorage` , `navigator`).
 - Event handlers (`onClick` , `onChange`).
- **Ví dụ:**

```
"use client";
import { useState } from 'react';
export default function InteractiveWelcome() {
  const [message, setMessage] = useState("Chào mừng!");
  return (
    <div>
      <p>{message}</p>
      <button onClick={() => setMessage("Đã click!")}>Click</button>
    </div>
  );
}
```

Khi nào sử dụng

- Tương tác người dùng (button, form).
- Quản lý state phía client.
- Sử dụng lifecycle effects (`useEffect`).
- Sử dụng browser APIs hoặc thư viện client-side (Redux, Context).
- Cần hooks chạy trên client.

Xử lý URL

- **Dynamic Routes:** Sử dụng hook `useParams` .

```
"use client";
import { useParams } from 'next/navigation';
export default function BlogIdDisplay() {
  const { blogId } = useParams();
}
```

```
return <p>ID bài viết: {blogId}</p>;
}
```

- **Search Params:** Sử dụng hook `useSearchParams` .

```
"use client";
import { useSearchParams } from 'next/navigation';
export default function InteractiveWelcome() {
  const searchParams = useSearchParams();
  const name = searchParams.get('name');
  return <p>Chào {name || 'Khách'}!</p>;
}
```

Mô hình kết hợp (Composition Patterns)

Quy tắc

- **SC có thể import/render CC:** Cách phổ biến để thêm tương tác.
- **CC import SC:** Có thể gây lỗi; thay vào đó, truyền SC làm prop `children` .

```
// ClientComponent.js
"use client";
export default function ClientComponent({ children }) {
  return <div>{children}</div>;
}

// ServerComponent.js
import ClientComponent from './ClientComponent';
export default async function ServerComponent() {
  return <ClientComponent><AnotherServerComponent /></ClientComponent>;
}
```

Props

- Props từ SC sang CC phải **serializable** (ví dụ: chuỗi, số, mảng, object đơn giản).
- **Non-serializable** props (hàm, Promise, DOM elements) gây lỗi.
- **Serializable Types:**
 - Nguyên thủy: `string` , `number` , `boolean` , `null` .
 - Phức tạp: Mảng và object thuần (nếu nội dung serializable).
- **Non-serializable:** Hàm, `Date` , `Set` , `Map` , `undefined` , `Symbol` , `BigInt` , DOM objects.

Kinh nghiệm

- Giữ component cha là SC để tối ưu hiệu suất.
- Chỉ dùng CC cho các component con cần tương tác.
- Truyền SC làm `children` cho CC để tránh lỗi import.

Luồng Render

Tải lần đầu (Initial Load)

1. Request gửi đến server.
2. Next.js render SC trên server, fetch dữ liệu.
3. CC được pre-render thành HTML trên server.
4. Server gửi HTML + RSC Payload.

5. Client hiển thị HTML ngay lập tức.
6. React hydrate CC để thêm tính tương tác.

Điều hướng phía Client

1. Người dùng click `<Link>` hoặc gọi `router.push()` .
2. Next.js yêu cầu component mới từ server.
3. Server render SC, gửi RSC Payload.
4. Client cập nhật DOM thông minh, giữ state của CC trong layout không đổi.

Khi nào SC re-render?

- Route thay đổi.
- `searchParams` thay đổi.
- Gọi `router.refresh()` .
- Server Action thực hiện thành công.
- Revalidation (ISR hoặc on-demand).

Nguyên tắc chung

- **Mặc định dùng SC:** Giảm JavaScript gửi xuống client.
- **Chỉ dùng CC khi cần:**
 - State, effects, browser APIs.
 - Tương tác người dùng (event handlers).
- Đẩy CC xuống sâu trong cây component.
- **Tối ưu:**
 - Fetch song song trong SC.
 - Đặt CC ở các vị trí cần tương tác.
 - Đảm bảo props serializable.
- **Hiệu suất:** SC giảm bundle size; CC hỗ trợ tương tác.